

Tutorial on numerical optimal control

Sébastien Kleff

Oct. 17, 2024

SESSION 2

In Session 1 : the continuous-time optimal control problem (OCP)

$$\begin{aligned} V(x_0) = \min_{u(\cdot)} \int_0^T \ell(x(t), u(t)) dt + \ell_T(x(T)) \\ \text{s.t. } \begin{cases} x(0) = x_0 \\ \dot{x}(t) = f(x(t), u(t)) & \forall t \in [0, T] \\ c(x(t), u(t)) \leq 0 & \forall t \in [0, T] \end{cases} \end{aligned} \quad (1)$$

was transcribed into a *discrete-time* OCP

$$\begin{aligned} V_0(x_0) = \min_{u_0, \dots, u_{T-1}} \sum_{k=0}^{T-1} \ell(x_k, u_k) + \ell_T(x_T) \\ \text{s.t. } \begin{cases} x_0 = \hat{x} \\ x_{k+1} = f(x_k, u_k) & \forall k \in \{0, T-1\} \\ c(x_k, u_k) \leq 0 & \forall k \in \{0, T-1\} \end{cases} \end{aligned} \quad (??)$$

which has now **finite-dimensional** decision variables.

The discrete-time OCP

$$\begin{aligned} V_0(x_0) = \min_{u_0, \dots, u_{T-1}} \quad & \sum_{k=0}^{T-1} \ell(x_k, u_k) + \ell_T(x_T) \\ \text{s.t.} \quad & \begin{cases} x_0 = \hat{x} \\ x_{k+1} = f(x_k, u_k) \\ c(x_k, u_k) \leq 0 \end{cases} \end{aligned} \quad (??)$$

is still intractable to solve globally.

But we said it could be solved **locally as a Nonlinear Program (NLP)**

$$\begin{aligned} \min_z \quad & L(z) \\ \text{s.t.} \quad & G(z) \leq 0 \end{aligned} \quad (??)$$

using standard numerical optimization techniques (see Nocedal)

The discrete-time OCP

$$\begin{aligned} V_0(x_0) = \min_{u_0, \dots, u_{T-1}} \sum_{k=0}^{T-1} \ell(x_k, u_k) + \ell_T(x_T) \quad (??) \\ \text{s.t.} \quad \begin{cases} x_0 = \hat{x} \\ x_{k+1} = f(x_k, u_k) \\ c(x_k, u_k) \leq 0 \end{cases} \end{aligned}$$

is still intractable to solve globally.

But we said it could be solved **locally as a Nonlinear Program (NLP)**

$$\begin{aligned} \min_z L(z) \quad (??) \\ \text{s.t.} \quad G(z) \leq 0 \end{aligned}$$

using standard numerical optimization techniques (see Nocedal)

Today I will show you how to solve NLPs !

Today, we will study the NLP

$$\begin{aligned} \min_z \quad & F(z) \\ \text{s.t.} \quad & G(z) = 0 \\ & H(z) \leq 0 \end{aligned} \tag{2}$$

Today, we will study the NLP

$$\begin{aligned} \min_z \quad & F(z) \\ \text{s.t.} \quad & G(z) = 0 \\ & H(z) \leq 0 \end{aligned} \tag{2}$$

Notations warning :

- The NLP cost is denoted by F (vs L last time)
- The NLP constraint is split into equality (G) and inequality (H)

Unconstrained optimization optimization

Consider the unconstrained optimization problem

$$\min_{z \in \mathbb{R}^n} F(z) \quad (3)$$

Unconstrained optimization optimization

Consider the unconstrained optimization problem

$$\min_{z \in \mathbb{R}^n} F(z) \quad (3)$$

Definition (Global minimizer)

z^* is called a global minimizer if for any $z \in \mathbb{R}^n$, $F(z^*) \leq F(z)$

Unconstrained optimization optimization

Consider the unconstrained optimization problem

$$\min_{z \in \mathbb{R}^n} F(z) \quad (3)$$

Definition (Global minimizer)

z^* is called a global minimizer if for any $z \in \mathbb{R}^n$, $F(z^*) \leq F(z)$

Definition (Local minimizer)

z^* is called a local minimizer if there exist a neighborhood $\mathcal{N}$ of z^* such that for any $z \in \mathcal{N}$, $F(z^*) \leq F(z)$

Unconstrained optimization optimization

Consider the unconstrained optimization problem

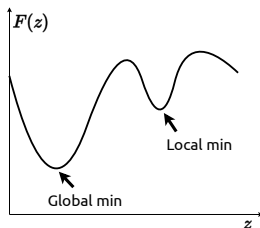
$$\min_{z \in \mathbb{R}^n} F(z) \quad (3)$$

Definition (Global minimizer)

z^* is called a global minimizer if for any $z \in \mathbb{R}^n$, $F(z^*) \leq F(z)$

Definition (Local minimizer)

z^* is called a local minimizer if there exist a neighborhood $\mathcal{N}$ of z^* such that for any $z \in \mathcal{N}$, $F(z^*) \leq F(z)$



Unconstrained optimization optimization

Consider the unconstrained optimization problem

$$\min_{z \in \mathbb{R}^n} F(z) \quad (3)$$

Definition (Global minimizer)

z^* is called a global minimizer if for any $z \in \mathbb{R}^n$, $F(z^*) \leq F(z)$

Definition (Local minimizer)

z^* is called a local minimizer if there exist a neighborhood $\mathcal{N}$ of z^* such that for any $z \in \mathcal{N}$, $F(z^*) \leq F(z)$

- Global minimizer $\implies$ local minimizer (converse not true)
- Global minimizers are often difficult to find

Unconstrained optimization

Consider the unconstrained optimization problem

$$\min_{z \in \mathbb{R}^n} F(z) \quad (3)$$

Definition (Global minimizer)

z^* is called a global minimizer if for any $z \in \mathbb{R}^n$, $F(z^*) \leq F(z)$

Definition (Local minimizer)

z^* is called a local minimizer if there exist a neighborhood $\mathcal{N}$ of z^* such that for any $z \in \mathcal{N}$, $F(z^*) \leq F(z)$

- Global minimizer $\implies$ local minimizer (converse not true)
- Global minimizers are often difficult to find

Theorem (First-order necessary conditions)

If z^ is a local minimizer then it is a stationary point, i.e. $\nabla F(z^*) = 0$.*

Unconstrained optimization

Consider the unconstrained optimization problem

$$\min_{z \in \mathbb{R}^n} F(z) \quad (3)$$

Definition (Global minimizer)

z^* is called a global minimizer if for any $z \in \mathbb{R}^n$, $F(z^*) \leq F(z)$

Definition (Local minimizer)

z^* is called a local minimizer if there exist a neighborhood $\mathcal{N}$ of z^* such that for any $z \in \mathcal{N}$, $F(z^*) \leq F(z)$

- Global minimizer $\implies$ local minimizer (converse not true)
- Global minimizers are often difficult to find

Theorem (First-order necessary conditions)

If z^ is a local minimizer then it is a stationary point, i.e. $\nabla F(z^*) = 0$.*

- We typically look for stationary points
- Most of the interesting results hold when F is convex

Unconstrained optimization optimization

We look for **local** minimizers by constructing a sequence of iterates $(z^k)_{k \in \mathbb{N}}$ converging to a local minimum of F

$$z^{k+1} = z^k + \alpha^k \Delta z^k$$

where $\Delta z^k \in \mathbb{R}^n$ is the **search direction** and $\alpha^k \geq 0$ is the **step size**.

Unconstrained optimization

We look for **local** minimizers by constructing a sequence of iterates $(z^k)_{k \in \mathbb{N}}$ converging to a local minimum of F

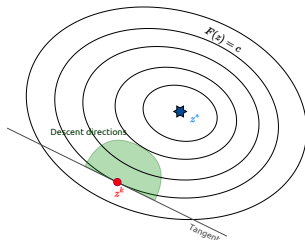
$$z^{k+1} = z^k + \alpha^k \Delta z^k$$

where $\Delta z^k \in \mathbb{R}^n$ is the **search direction** and $\alpha^k \geq 0$ is the **step size**.

Definition (Descent direction)

The search direction Δz^k is a valid descent direction if

$$\nabla F(z^k)^\top \Delta z^k < 0$$



Unconstrained optimization

We look for **local** minimizers by constructing a sequence of iterates $(z^k)_{k \in \mathbb{N}}$ converging to a local minimum of F

$$z^{k+1} = z^k + \alpha^k \Delta z^k$$

where $\Delta z^k \in \mathbb{R}^n$ is the **search direction** and $\alpha^k \geq 0$ is the **step size**.

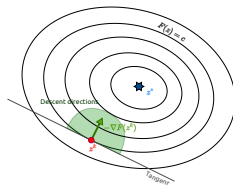
Definition (Descent direction)

The search direction Δz^k is a valid descent direction if

$$\nabla F(z^k)^\top \Delta z^k < 0$$

Example (Steepest descent)

The steepest descent direction $\Delta z = -\nabla F(z)$ is a valid descent direction.



Unconstrained optimization

We look for **local** minimizers by constructing a sequence of iterates $(z^k)_{k \in \mathbb{N}}$ converging to a local minimum of F

$$z^{k+1} = z^k + \alpha^k \Delta z^k$$

where $\Delta z^k \in \mathbb{R}^n$ is the **search direction** and $\alpha^k \geq 0$ is the **step size**.

Definition (Descent direction)

The search direction Δz^k is a valid descent direction if

$$\nabla F(z^k)^\top \Delta z^k < 0$$

Example (Steepest descent)

The steepest descent direction $\Delta z = -\nabla F(z)$ is a valid descent direction.

Example (Newton's method)

The Newton direction defined by

$$\Delta z = -\nabla^2 F(z)^{-1} \nabla F(z) \tag{4}$$

is a valid descent direction (for $\nabla^2 F(z) \succ 0$).

Unconstrained optimization optimization

Once a valid descent direction Δz has been chosen, we need to make sure that the step corresponds to an actual decrease in the objective F , i.e. find α s.t.

$$F(z + \alpha \Delta z) < F(z)$$

Unconstrained optimization optimization

Once a valid descent direction Δz has been chosen, we need to make sure that the step corresponds to an actual decrease in the objective F , i.e. find α s.t.

$$F(z + \alpha \Delta z) < F(z)$$

Example (Exact line search)

Select the step size that benefits the most from Δz

$$\alpha = \arg \min_{\alpha > 0} F(z + \alpha \Delta z)$$

Unconstrained optimization optimization

Once a valid descent direction Δz has been chosen, we need to make sure that the step corresponds to an actual decrease in the objective F , i.e. find α s.t.

$$F(z + \alpha \Delta z) < F(z)$$

Example (Exact line search)

Select the step size that benefits the most from Δz

$$\alpha = \arg \min_{\alpha > 0} F(z + \alpha \Delta z)$$

Example (Backtracking line search (BLTS))

Try and reduce α until some "sufficient decrease" condition is met

Result: step size α

Choose $\alpha, \beta, \gamma \in]0, 1[$

while $F(z + \alpha \Delta z) > F(z) + \alpha \beta \nabla F(z)^\top \Delta z$ **do**
 $\alpha \leftarrow \gamma \alpha$

Numerical optimization: Steepest descent

Gradient descent minimizes the 1st-order Taylor expansion of F around the current iterate

$$F(z + \Delta z) \approx F(z) + \nabla F(z)^\top \Delta z + o(\|\Delta z\|^2)$$

by searching along the negative gradient $\Delta z = -\nabla F(z)$

Numerical optimization: Steepest descent

Gradient descent minimizes the 1st-order Taylor expansion of F around the current iterate

$$F(z + \Delta z) \approx F(z) + \nabla F(z)^\top \Delta z + o(\|\Delta z\|^2)$$

by searching along the negative gradient $\Delta z = -\nabla F(z)$

Theorem (Global linear convergence of gradient descent)

When f is strongly convex, gradient descent with BLTS converges linearly to the global minimum $F^ \in \mathbb{R}$, i.e. there exist $\eta > 0$ s.t.*

$$F(z^{k+1}) - F^* \leq \eta \left(F(z^{k+1}) - F^* \right)$$

Numerical optimization: Steepest descent

Gradient descent minimizes the 1st-order Taylor expansion of F around the current iterate

$$F(z + \Delta z) \approx F(z) + \nabla F(z)^\top \Delta z + o(\|\Delta z\|^2)$$

by searching along the negative gradient $\Delta z = -\nabla F(z)$

Theorem (Global linear convergence of gradient descent)

When f is strongly convex, gradient descent with BLTS converges linearly to the global minimum $F^ \in \mathbb{R}$, i.e. there exist $\eta > 0$ s.t.*

$$F(z^{k+1}) - F^* \leq \eta \left(F(z^{k+1}) - F^* \right)$$

- Global linear convergence
- The linear convergence rate η depends on the conditioning $\nabla^2 F(z)$
- In practice, convergence can be ridiculously slow
- Play with the code : `mpc_tutorial/optimization/unconstrained.py`

Numerical optimization: Newton direction

Newton method minimizes the 2^{nd} -order Taylor expansion

$$F(z + \Delta z) \approx F(z) + \nabla F(z)^\top \Delta z + \frac{1}{2} \Delta z^\top \nabla^2 F(z) \Delta z + o(\|\Delta z\|^3)$$

by searching along $\Delta z = -(\nabla^2 F(z))^{-1} \nabla F(z)$

Numerical optimization: Newton direction

Newton method minimizes the 2^{nd} -order Taylor expansion

$$F(z + \Delta z) \approx F(z) + \nabla F(z)^\top \Delta z + \frac{1}{2} \Delta z^\top \nabla^2 F(z) \Delta z + o(\|\Delta z\|^3)$$

by searching along $\Delta z = -(\nabla^2 F(z))^{-1} \nabla F(z)$

Theorem (Local quadratic convergence of Newton)

When f is strongly convex, Newton method with BLTS converges quadratically to $F^ \in \mathbb{R}$, i.e. there exist $\eta > 0$ s.t.*

$$\|\nabla F(z^{k+1})\| \leq \left(\eta \|\nabla F(z^{k+1})\| \right)^2$$

*provided that z^0 is close enough to z^**

Numerical optimization: Newton direction

Newton method minimizes the 2^{nd} -order Taylor expansion

$$F(z + \Delta z) \approx F(z) + \nabla F(z)^\top \Delta z + \frac{1}{2} \Delta z^\top \nabla^2 F(z) \Delta z + o(\|\Delta z\|^3)$$

by searching along $\Delta z = -(\nabla^2 F(z))^{-1} \nabla F(z)$

Theorem (Local quadratic convergence of Newton)

When f is strongly convex, Newton method with BLTS converges quadratically to $F^ \in \mathbb{R}$, i.e. there exist $\eta > 0$ s.t.*

$$\|\nabla F(z^{k+1})\| \leq \left(\eta \|\nabla F(z^{k+1})\| \right)^2$$

*provided that z^0 is close enough to z^**

- Local quadratic convergence : z^0 must be "close enough" to z^*
- Convergence is *much faster* than gradient descent
- $\nabla^2 F(z)$ is expensive to compute and must be $\succ 0$
- Play with the code : `mpc_tutorial/optimization/unconstrained.py`

Unconstrained optimization (recap)

In order to solve the unconstrained minimization problem

$$\min_{z \in \mathbb{R}^n} F(z)$$

Unconstrained optimization (recap)

In order to solve the unconstrained minimization problem

$$\min_{z \in \mathbb{R}^n} F(z)$$

we are looking for **local minimizers** of F by iteratively computing a **descent direction** and searching for a sufficient decrease along this direction.

Unconstrained optimization (recap)

In order to solve the unconstrained minimization problem

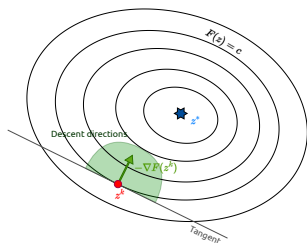
$$\min_{z \in \mathbb{R}^n} F(z)$$

we are looking for **local minimizers** of F by iteratively computing a **descent direction** and searching for a sufficient decrease along this direction.

Gradient descent

1st-order method leveraging ∇F

Global linear convergence rate



Unconstrained optimization (recap)

In order to solve the unconstrained minimization problem

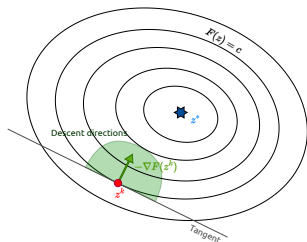
$$\min_{z \in \mathbb{R}^n} F(z)$$

we are looking for **local minimizers** of F by iteratively computing a **descent direction** and searching for a sufficient decrease along this direction.

Gradient descent

1st-order method leveraging ∇F

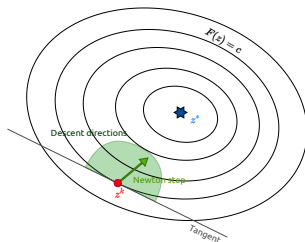
Global linear convergence rate



Newton method

2nd-order method leveraging $\nabla F, \nabla^2 F$

Local quadratic convergence rate



Unconstrained optimization (recap)

In order to solve the unconstrained minimization problem

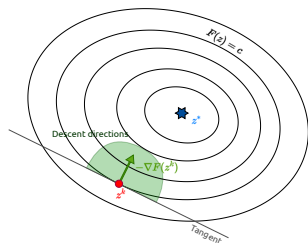
$$\min_{z \in \mathbb{R}^n} F(z)$$

we are looking for **local minimizers** of F by iteratively computing a **descent direction** and searching for a sufficient decrease along this direction.

Gradient descent

1^{st} -order method leveraging ∇F

Global linear convergence rate

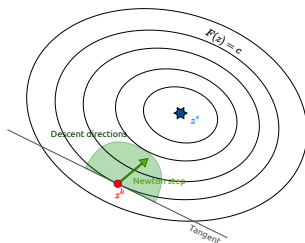


Example

Newton method

2^{nd} -order method leveraging $\nabla F, \nabla^2 F$

Local quadratic convergence rate



Unconstrained optimization (recap)

In order to solve the unconstrained minimization problem

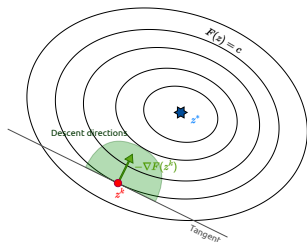
$$\min_{z \in \mathbb{R}^n} F(z)$$

we are looking for **local minimizers** of F by iteratively computing a **descent direction** and searching for a sufficient decrease along this direction.

Gradient descent

1st-order method leveraging ∇F

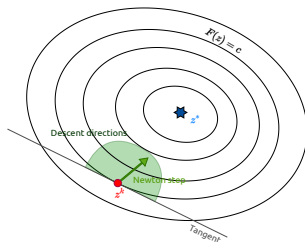
Global linear convergence rate



Newton method

2nd-order method leveraging $\nabla F, \nabla^2 F$

Local quadratic convergence rate



What about constraints?

Constrained optimization: Problem

Consider now the problem with **constraints** $G : \mathbb{R}^n \mapsto \mathbb{R}^{n_e}, H : \mathbb{R}^n \mapsto \mathbb{R}^{n_i}$

$$\begin{aligned} \min_{z \in \mathbb{R}^n} \quad & F(z) \\ \text{s.t.} \quad & G(z) = 0 \\ & H(z) \leq 0 \end{aligned}$$

Constrained optimization: Problem

Consider now the problem with **constraints** $G : \mathbb{R}^n \mapsto \mathbb{R}^{n_e}, H : \mathbb{R}^n \mapsto \mathbb{R}^{n_i}$

$$\begin{aligned} \min_{z \in \mathbb{R}^n} \quad & F(z) \\ \text{s.t.} \quad & G(z) = 0 \\ & H(z) \leq 0 \end{aligned}$$

Definition (Feasible set)

The feasible set is defined as

$$\Omega = \{z \in \mathbb{R}^n \mid G(z) = 0 \wedge H(z) \leq 0\}$$

A minimizer z^* is a feasible point $z^* \in \Omega$ that minimizes F .

Constrained optimization: Problem

Consider now the problem with **constraints** $G : \mathbb{R}^n \mapsto \mathbb{R}^{n_e}, H : \mathbb{R}^n \mapsto \mathbb{R}^{n_i}$

$$\begin{aligned} \min_{z \in \mathbb{R}^n} \quad & F(z) \\ \text{s.t.} \quad & G(z) = 0 \\ & H(z) \leq 0 \end{aligned}$$

Definition (Feasible set)

The feasible set is defined as

$$\Omega = \{z \in \mathbb{R}^n \mid G(z) = 0 \wedge H(z) \leq 0\}$$

A minimizer z^* is a feasible point $z^* \in \Omega$ that minimizes F .

- Same notions of global vs local minimizers
- We are usually looking for a *local* solution $z^* \in \Omega$
- Easier (i.e. many results) when F is convex *and* G, H are linear

Constrained optimization: Problem

Consider now the problem with **constraints** $G : \mathbb{R}^n \mapsto \mathbb{R}^{n_e}, H : \mathbb{R}^n \mapsto \mathbb{R}^{n_i}$

$$\begin{aligned} \min_{z \in \mathbb{R}^n} \quad & F(z) \\ \text{s.t.} \quad & G(z) = 0 \\ & H(z) \leq 0 \end{aligned}$$

Definition (Feasible set)

The feasible set is defined as

$$\Omega = \{z \in \mathbb{R}^n \mid G(z) = 0 \wedge H(z) \leq 0\}$$

A minimizer z^* is a feasible point $z^* \in \Omega$ that minimizes F .

- Same notions of global vs local minimizers
- We are usually looking for a *local* solution $z^* \in \Omega$
- Easier (i.e. many results) when F is convex *and* G, H are linear

Like in the unconstrained case, we will look for **local optima** by enforcing some first-order **necessary** conditions

Constrained optimization: KKT conditions

Definition (Lagrangian)

We define the Lagrangian as

$$\mathcal{L}(z, \lambda, \mu) = F(z) + \lambda^\top G(z) + \mu^\top H(z)$$

where $(\lambda, \mu) \in \mathbb{R}^{n_e} \times \mathbb{R}^{n_i}$ are the Lagrange multipliers associated (G, H) .
 (λ, μ) are also called the *dual variables* and z the *primal variable*.

Constrained optimization: KKT conditions

Definition (Lagrangian)

We define the Lagrangian as

$$\mathcal{L}(z, \lambda, \mu) = F(z) + \lambda^\top G(z) + \mu^\top H(z)$$

where $(\lambda, \mu) \in \mathbb{R}^{n_e} \times \mathbb{R}^{n_i}$ are the Lagrange multipliers associated (G, H) .
 (λ, μ) are also called the *dual variables* and z the *primal variable*.

Theorem (First-order necessary conditions for optimality)

If z^* be a local minimizer (+ some assumptions), then there exist $\lambda^* \in \mathbb{R}^{n_e}$ and $\mu^* \in \mathbb{R}^{n_i}$ such that the Karush-Kuhn-Tucker (KKT) conditions hold

$$\nabla_z \mathcal{L}(z^*, \lambda^*, \mu^*) = 0 \quad \text{Stationarity} \quad (5)$$

$$G(z^*) = 0 \wedge H(z^*) \leq 0 \quad \text{Primal feasibility} \quad (6)$$

$$\mu_i \geq 0 \quad \text{Dual feasibility} \quad (7)$$

$$\mu_i H_i(z^*) = 0 \quad \text{Complementary slackness} \quad (8)$$

- In the unconstrained case we were looking for stationary points
- We are now looking for **KKT points**

Constrained optimization: Newton method

Let's focus on the equality-constrained problem first (no H)

$$\begin{bmatrix} \nabla_z \mathcal{L}(z, \lambda) \\ G(z) \end{bmatrix} = 0 \quad (9)$$

Constrained optimization: Newton method

Let's focus on the equality-constrained problem first (no H)

$$\begin{bmatrix} \nabla_z \mathcal{L}(z, \lambda) \\ G(z) \end{bmatrix} = 0 \quad (9)$$

Now recall a fundamental result from undergrad analysis.

Theorem (Newton-Raphson method for nonlinear equations)

We can find a root of the nonlinear equation

$$r(w) = 0$$

iteratively using the Newton-Raphson algorithm

Result: *Root w*

while $\|r(w)\| > tol$ **do**

Compute the step direction Δw s.t. $\nabla r(w)^\top \Delta w = -r(w)$

Take the step $w \leftarrow w + \Delta w$

Constrained optimization: Newton method

Let's focus on the equality-constrained problem first (no H)

$$\begin{bmatrix} \nabla_z \mathcal{L}(z, \lambda) \\ G(z) \end{bmatrix} = 0 \quad (9)$$

Now recall a fundamental result from undergrad analysis.

Theorem (Newton-Raphson method for nonlinear equations)

We can find a root of the nonlinear equation

$$r(w) = 0$$

iteratively using the Newton-Raphson algorithm

Result: Root w

while $\|r(w)\| > \text{tol}$ **do**

 Compute the step direction Δw s.t. $\nabla r(w)^\top \Delta w = -r(w)$

 Take the step $w \leftarrow w + \Delta w$

Example : Newton-Raphson to solve $r(w) = 0$ with $r(w) = w^4 - 3w^3 + 2$

Play with it : `mpc_tutorial/optimization/newton_raphson.py`

Constrained optimization: Newton method

Let's focus on the equality-constrained problem first (no H)

$$\begin{bmatrix} \nabla_z \mathcal{L}(z, \lambda) \\ G(z) \end{bmatrix} = 0 \quad (9)$$

Idea: Apply Newton-Raphson to find a root of (9)

Solve $r(z, \lambda) = 0$ with

$$r(z, \lambda) = \begin{bmatrix} \nabla_z \mathcal{L}(z, \lambda) \\ G(z) \end{bmatrix}$$

Constrained optimization: Newton method

Let's focus on the equality-constrained problem first (no H)

$$\begin{bmatrix} \nabla_z \mathcal{L}(z, \lambda) \\ G(z) \end{bmatrix} = 0 \quad (9)$$

The Newton-Raphson step $\Delta w \triangleq (\Delta z, \Delta \lambda)$ is given by solving

$$\underbrace{\begin{bmatrix} \nabla_{zz}^2 \mathcal{L}(z, \lambda) & \nabla G(z) \\ \nabla G(z)^\top & 0 \end{bmatrix}}_{\text{"}\nabla r(w)^\top\text{"}} \underbrace{\begin{bmatrix} \Delta z \\ \Delta \lambda \end{bmatrix}}_{\text{"}\Delta w\text{"}} = - \underbrace{\begin{bmatrix} \nabla_z \mathcal{L}(z, \lambda) \\ \nabla G(z) \end{bmatrix}}_{\text{"}r(w)\text{"}} \quad (10)$$

and it is called the **Newton direction**

Constrained optimization: Newton method

Let's focus on the equality-constrained problem first (no H)

$$\begin{bmatrix} \nabla_z \mathcal{L}(z, \lambda) \\ G(z) \end{bmatrix} = 0 \quad (9)$$

The Newton-Raphson step $\Delta w \triangleq (\Delta z, \Delta \lambda)$ is given by solving

$$\underbrace{\begin{bmatrix} \nabla_{zz}^2 \mathcal{L}(z, \lambda) & \nabla G(z) \\ \nabla G(z)^\top & 0 \end{bmatrix}}_{\text{"}\nabla r(w)^\top\text{"}} \underbrace{\begin{bmatrix} \Delta z \\ \Delta \lambda \end{bmatrix}}_{\text{"}\Delta w\text{"}} = - \underbrace{\begin{bmatrix} \nabla_z \mathcal{L}(z, \lambda) \\ \nabla G(z) \end{bmatrix}}_{\text{"}r(w)\text{"}} \quad (10)$$

and it is called the **Newton direction**

The primal-dual variables (z, λ) are then updated

$$\begin{bmatrix} z^{k+1} \\ \lambda^{k+1} \end{bmatrix} = \begin{bmatrix} z^k \\ \lambda^k \end{bmatrix} + \begin{bmatrix} \Delta z \\ \Delta \lambda \end{bmatrix} \quad (11)$$

We need $\nabla_{zz}^2 \mathcal{L}(z, \lambda) \succ 0$ and $\nabla G(z)$ full row rank

Constrained optimization: Newton method

Let's focus on the equality-constrained problem first (no H)

$$\begin{bmatrix} \nabla_z \mathcal{L}(z, \lambda) \\ G(z) \end{bmatrix} = 0 \quad (9)$$

The Newton-Raphson step $\Delta w \triangleq (\Delta z, \Delta \lambda)$ is given by solving

$$\underbrace{\begin{bmatrix} \nabla_{zz}^2 \mathcal{L}(z, \lambda) & \nabla G(z) \\ \nabla G(z)^\top & 0 \end{bmatrix}}_{\text{"}\nabla r(w)^\top\text{"}} \underbrace{\begin{bmatrix} \Delta z \\ \Delta \lambda \end{bmatrix}}_{\text{"}\Delta w\text{"}} = - \underbrace{\begin{bmatrix} \nabla_z \mathcal{L}(z, \lambda) \\ \nabla G(z) \end{bmatrix}}_{\text{"}r(w)\text{"}} \quad (10)$$

and it is called the **Newton direction**

The primal-dual variables (z, λ) are then updated

$$\begin{bmatrix} z^{k+1} \\ \lambda^{k+1} \end{bmatrix} = \begin{bmatrix} z^k \\ \lambda^k \end{bmatrix} + \begin{bmatrix} \Delta z \\ \Delta \lambda \end{bmatrix} \quad (11)$$

We need $\nabla_{zz}^2 \mathcal{L}(z, \lambda) \succ 0$ and $\nabla G(z)$ full row rank

Example : $F(z) = z^\top A z$ and $G(z) = z^\top B z - c$

Play with it : [mpc_tutorial/optimization/constrained.py](#)

Constrained optimization: Newton method

Let's focus on the equality-constrained problem first (no H)

$$\begin{bmatrix} \nabla_z \mathcal{L}(z, \lambda) \\ G(z) \end{bmatrix} = 0 \quad (9)$$

The Newton-Raphson step $\Delta w \triangleq (\Delta z, \Delta \lambda)$ is given by solving

$$\underbrace{\begin{bmatrix} \nabla_{zz}^2 \mathcal{L}(z, \lambda) & \nabla G(z) \\ \nabla G(z)^\top & 0 \end{bmatrix}}_{\text{"}\nabla r(w)^\top\text{"}} \underbrace{\begin{bmatrix} \Delta z \\ \Delta \lambda \end{bmatrix}}_{\text{"}\Delta w\text{"}} = - \underbrace{\begin{bmatrix} \nabla_z \mathcal{L}(z, \lambda) \\ \nabla G(z) \end{bmatrix}}_{\text{"}r(w)\text{"}} \quad (10)$$

and it is called the **Newton direction**

The primal-dual variables (z, λ) are then updated

$$\begin{bmatrix} z^{k+1} \\ \lambda^{k+1} \end{bmatrix} = \begin{bmatrix} z^k \\ \lambda^k \end{bmatrix} + \begin{bmatrix} \Delta z \\ \Delta \lambda \end{bmatrix} \quad (11)$$

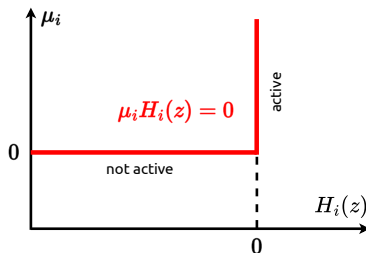
We need $\nabla_{zz}^2 \mathcal{L}(z, \lambda) \succ 0$ and $\nabla G(z)$ full row rank

What about inequalities now ?

Constrained optimization: Inequalities

The complementary slackness condition $\mu_i H_i(z^*) = 0$ is **non-smooth**

We **cannot** apply Newton-Raphson directly in the presence of inequalities



Constrained optimization: Inequalities

Key: re-interpret the Newton method for equality-constrained problems

Theorem (Quadratic model interpretation)

*The Newton-Raphson direction is **also** given by solving a linear-quadratic approximation of the original NLP*

$$\begin{aligned} \min_{\Delta z} \quad & \frac{1}{2} \Delta z^\top \nabla_{zz} \mathcal{L}(z, \lambda) \Delta z + \nabla F(z)^\top \Delta z \\ \text{s.t.} \quad & \nabla G(z)^\top \Delta z + G(z) = 0 \quad (\lambda^{QP}) \end{aligned} \tag{12}$$

In particular we have $\lambda^{QP} = \lambda^{k+1}$

Constrained optimization: Inequalities

Key: re-interpret the Newton method for equality-constrained problems

Theorem (Quadratic model interpretation)

*The Newton-Raphson direction is **also** given by solving a linear-quadratic approximation of the original NLP*

$$\begin{aligned} \min_{\Delta z} \quad & \frac{1}{2} \Delta z^\top \nabla_{zz} \mathcal{L}(z, \lambda) \Delta z + \nabla F(z)^\top \Delta z \\ \text{s.t.} \quad & \nabla G(z)^\top \Delta z + G(z) = 0 \quad (\lambda^{QP}) \end{aligned} \tag{12}$$

In particular we have $\lambda^{QP} = \lambda^{k+1}$

Constrained optimization: Inequalities

Key: re-interpret the Newton method for equality-constrained problems

Theorem (Quadratic model interpretation)

*The Newton-Raphson direction is **also** given by solving a linear-quadratic approximation of the original NLP*

$$\begin{aligned} \min_{\Delta z} \quad & \frac{1}{2} \Delta z^\top \nabla_{zz} \mathcal{L}(z, \lambda) \Delta z + \nabla F(z)^\top \Delta z \\ \text{s.t.} \quad & \nabla G(z)^\top \Delta z + G(z) = 0 \quad (\lambda^{QP}) \end{aligned} \tag{12}$$

In particular we have $\lambda^{QP} = \lambda^{k+1}$

We were just solving a QP !

Constrained optimization: Inequalities

Key: re-interpret the Newton method for equality-constrained problems

Theorem (Quadratic model interpretation)

*The Newton-Raphson direction is **also** given by solving a linear-quadratic approximation of the original NLP*

$$\begin{aligned} \min_{\Delta z} \quad & \frac{1}{2} \Delta z^\top \nabla_{zz} \mathcal{L}(z, \lambda) \Delta z + \nabla F(z)^\top \Delta z \\ \text{s.t.} \quad & \nabla G(z)^\top \Delta z + G(z) = 0 \quad (\lambda^{QP}) \end{aligned} \quad (12)$$

In particular we have $\lambda^{QP} = \lambda^{k+1}$

We were just solving a QP !

Based on this key observation, there are 2 main ways to handle inequalities :

- Interior Point (IP) : "Get rid of inequality"
- Sequential-Quadratic Programming (SQP) : "Give it to the QP"

Constrained optimization: SQP

Idea : relegate the inequalities to the QP

Idea : relegate the inequalities to the QP

We are solving a **sequence of linear-quadratic approximations** of the original NLP around each iterate (z^k, λ^k)

$$\begin{aligned} \min_{\Delta z} \quad & \frac{1}{2} \Delta z^\top \nabla_{zz} \mathcal{L}(z^k, \lambda^k, \mu^k) \Delta z + \nabla F(z^k)^\top \Delta z \\ \text{s.t.} \quad & \nabla G(z^k)^\top \Delta z + G(z^k) = 0 \\ & \nabla H(z^k)^\top \Delta z + H(z^k) \leq 0 \end{aligned} \tag{13}$$

Idea : relegate the inequalities to the QP

We are solving a **sequence of linear-quadratic approximations** of the original NLP around each iterate (z^k, λ^k)

$$\begin{aligned} \min_{\Delta z} \quad & \frac{1}{2} \Delta z^\top \nabla_{zz} \mathcal{L}(z^k, \lambda^k, \mu^k) \Delta z + \nabla F(z^k)^\top \Delta z \\ \text{s.t.} \quad & \nabla G(z^k)^\top \Delta z + G(z^k) = 0 \\ & \nabla H(z^k)^\top \Delta z + H(z^k) \leq 0 \end{aligned} \tag{13}$$

Use any QP solver you like to compute the Newton direction $(\Delta z, \Delta \lambda, \Delta \mu)$

Idea : relegate the inequalities to the QP

We are solving a **sequence of linear-quadratic approximations** of the original NLP around each iterate (z^k, λ^k)

$$\begin{aligned} \min_{\Delta z} \quad & \frac{1}{2} \Delta z^\top \nabla_{zz} \mathcal{L}(z^k, \lambda^k, \mu^k) \Delta z + \nabla F(z^k)^\top \Delta z \\ \text{s.t.} \quad & \nabla G(z^k)^\top \Delta z + G(z^k) = 0 \\ & \nabla H(z^k)^\top \Delta z + H(z^k) \leq 0 \end{aligned} \tag{13}$$

Use any QP solver you like to compute the Newton direction $(\Delta z, \Delta \lambda, \Delta \mu)$

Typically perform a **line-search** using a *merit function* [Nocedal]

$$z^{k+1} = z^k + \alpha \Delta z$$

$$\lambda^{k+1} = \lambda^k + \alpha \Delta \lambda$$

$$\mu^{k+1} = \mu^k + \alpha \Delta \mu$$

Idea : relegate the inequalities to the QP

We are solving a **sequence of linear-quadratic approximations** of the original NLP around each iterate (z^k, λ^k)

$$\begin{aligned} \min_{\Delta z} \quad & \frac{1}{2} \Delta z^\top \nabla_{zz} \mathcal{L}(z^k, \lambda^k, \mu^k) \Delta z + \nabla F(z^k)^\top \Delta z \\ \text{s.t.} \quad & \nabla G(z^k)^\top \Delta z + G(z^k) = 0 \\ & \nabla H(z^k)^\top \Delta z + H(z^k) \leq 0 \end{aligned} \tag{13}$$

Use any QP solver you like to compute the Newton direction $(\Delta z, \Delta \lambda, \Delta \mu)$

Typically perform a **line-search** using a *merit function* [Nocedal]

$$z^{k+1} = z^k + \alpha \Delta z$$

$$\lambda^{k+1} = \lambda^k + \alpha \Delta \lambda$$

$$\mu^{k+1} = \mu^k + \alpha \Delta \mu$$

SQP has local quadratic convergence !

Constrained optimization: Recap

Key take-aways :

- Constrained optimization is much harder

Constrained optimization: Recap

Key take-aways :

- Constrained optimization is much harder
- KKT conditions are the 1st-order necessary conditions for optimality

Constrained optimization: Recap

Key take-aways :

- Constrained optimization is much harder
- KKT conditions are the 1^{st} -order necessary conditions for optimality
- We can solve equality-constrained problems using Newton's method

Constrained optimization: Recap

Key take-aways :

- Constrained optimization is much harder
- KKT conditions are the 1^{st} -order necessary conditions for optimality
- We can solve equality-constrained problems using Newton's method
- This is equivalent to solving a sequence of equality-constrained QPs

Constrained optimization: Recap

Key take-aways :

- Constrained optimization is much harder
- KKT conditions are the 1st-order necessary conditions for optimality
- We can solve equality-constrained problems using Newton's method
- This is equivalent to solving a sequence of equality-constrained QPs
- Inequality constraints are handled based on this idea

Constrained optimization: Recap

Key take-aways :

- Constrained optimization is much harder
- KKT conditions are the 1st-order necessary conditions for optimality
- We can solve equality-constrained problems using Newton's method
- This is equivalent to solving a sequence of equality-constrained QPs
- Inequality constraints are handled based on this idea
- In particular, SQP solves a sequence of inequality-constrained QPs

Constrained optimization: Recap

Key take-aways :

- Constrained optimization is much harder
- KKT conditions are the 1st-order necessary conditions for optimality
- We can solve equality-constrained problems using Newton's method
- This is equivalent to solving a sequence of equality-constrained QPs
- Inequality constraints are handled based on this idea
- In particular, SQP solves a sequence of inequality-constrained QPs
- SQP converges quadratically to KKT points

This was a long detour from our original problem

Now how to use SQP to solve our discrete-time OCP ?

In Session 3 we will address 2 questions

- ① How to apply SQP to the solve the OCP ? i.e. what is z in terms of the OCP decision variables u_k, x_k ?
 - Single-shooting $z \triangleq (u_0, \dots, u_{T-1})$. Only controls are decision variables
 - Multiple-shooting $z \triangleq (x_0, u_0, \dots, x_{T-1}, u_{T-1}, x_T)$. Both controls *and* states are decisions variables.
- ② How to solve the SQP (2) *efficiently* by exploiting the underlying structure of the OCP ?
- ③ What is Model-Predictive Control (MPC) and how to achieve it in practice ?